Name: Jashruth K A **SRN: PES2UG24AM069**

Course: Computer Networks **Course Code: UE24CS252B**

Project Title: SDN Based Access Control System

Table of Contents:

1. **Problem Statement**
2. **SDN Architecture**
3. **Mininet Topology Design**
4. **Controller Implementation**
5. **Flow Rule Management**
6. **Functionality Implementation**
7. **Performance Evaluation**
8. **Python Scripts**
9. **Screenshots of Demo**
10. **GitHub Repository Link**
11. **Conclusion**

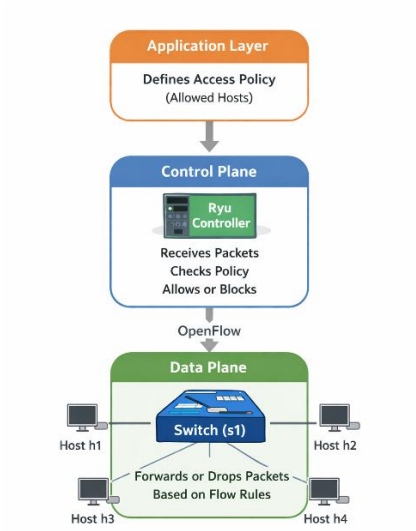
Problem Statement:

Implement an SDN-based access control system using Mininet and Ryu controller that allows only authorized hosts (h1 & h2) to communicate within the network. Unauthorized hosts (h3 & h4) are silently blocked using dynamic OpenFlow flow rules.

SDN Architecture: Software Defined Networking (SDN) separates the **control plane** and **data plane** for centralized network management.

- **Application Layer:** Implements access control logic (whitelist of authorized hosts)
- **Control Plane (Ryu Controller):** Decides whether to allow or block traffic and installs flow rules.
- **Data Plane (Open vSwitch):** Executes the flow rules to forward or drop packets.

When a packet arrives at the switch without a rule, it is sent to the controller. The controller checks the policy and installs an appropriate flow rule.



This architecture enables **dynamic control, flexibility, and improved network security**.

<u>Mininet Topology Design:</u> The network topology is implemented using Mininet and consists of **four hosts (h1–h4), one switch (s1), and a Ryu controller**.

Each host is assigned a unique IP address, and all hosts are connected to a single switch. The switch communicates with the Ryu controller for decision-making.

- **Authorized hosts:** h1, h2
- **Unauthorized hosts:** h3, h4

The topology is defined in topology.py and is used to simulate and test access control by allowing communication only between authorized hosts while blocking others.

<u>Controller implementation:</u> The controller is implemented using the **Ryu SDN framework** in access_control.py.

It listens for **packet_in events** from the switch and checks the source MAC address against a predefined whitelist.

- If the host is authorized → a **forward (allow) rule** is installed
- If the host is unauthorized → a **drop rule** is installed

The controller dynamically updates flow tables, ensuring only permitted communication occurs in the network.

Flow Rule Management: Flow rule management is handled by the Ryu controller using OpenFlow.

When a packet arrives at the switch without a matching rule, it is sent to the controller. The controller checks the source MAC address against the whitelist and installs appropriate flow rules:

- **Authorized hosts:** Forwarding rules are installed to allow communication
- **Unauthorized hosts:** Drop rules are installed to block traffic

These rules are dynamically added to the switch's flow table, ensuring efficient and consistent enforcement of access control policies

Functionality implementation: The system implements a **whitelist-based access control mechanism** where only authorized hosts are allowed to communicate.

When traffic is generated:

- Communication between authorized hosts (h1 and h2) is allowed
- Any traffic involving unauthorized hosts (h3 and h4) is blocked

The controller dynamically enforces these rules, ensuring that only permitted interactions occur within the network.

Performance Evaluation: The performance of the system is evaluated using ping and iperf tests.

- **Authorized communication (h1 ↔ h2):** Successful with 0% packet loss and high throughput
- **Unauthorized communication (h3/h4):** Blocked with 100% packet loss

The results confirm that the access control mechanism works correctly and maintains consistent policy enforcement.

Python Script: The project includes two main Python scripts:

- topology.py: Defines the Mininet topology with hosts, switch, and controller connection
- access_control.py: Implements the Ryu controller logic for access control and flow rule management

These scripts work together to simulate the network and enforce the access control policies.

Screenshot of Demo:

1. Mininet Topology Launched

```
*** Adding switch
*** Adding hosts
*** Adding links
*** Starting network
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
c0
*** Starting 1 switches
s1 ...

*** Hosts configured:
  h1 - 10.0.0.1 (AUTHORIZED)
  h2 - 10.0.0.2 (AUTHORIZED)
  h3 - 10.0.0.3 (UNAUTHORIZED)
  h4 - 10.0.0.4 (UNAUTHORIZED)

*** Opening Mininet CLI
*** Starting CLI:
mininet> h1 ping -c 4 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=4.84 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.071 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.055 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.070 ms

--- 10.0.0.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3071ms
rtt min/avg/max/mdev = 0.055/1.258/4.836/2.065 ms
mininet>
```

2. Ryu Controller Started

```
jash@Ubuntu:~$ cd CN-OP
jash@Ubuntu:~/CN-OP$ nano topology.py
jash@Ubuntu:~/CN-OP$ nano access_control.py
jash@Ubuntu:~/CN-OP$ source ~/ryu-env39/bin/activate
(ryu-env39) jash@Ubuntu:~/CN-OP$ ryu-manager access_control.py
loading app access_control.py
loading app ryu.controller.ofp_handler
instantiating app access_control.py of AccessControlController
=== SDN Access Control System Started ===
Whitelisted pairs: h1 <-> h2
All other traffic will be BLOCKED
instantiating app ryu.controller.ofp_handler of OFPHandler
Switch 1 connected
ALLOWED: 00:00:00:00:00:02 -> 00:00:00:00:00:01 (authorized)
ALLOWED: 00:00:00:00:00:01 -> 00:00:00:00:00:02 (authorized)
```

3. H1 <-> H2 Ping Success

```
mininet> h1 ping -c 4 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=3.65 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.068 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.063 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.061 ms

--- 10.0.0.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3066ms
rtt min/avg/max/mdev = 0.061/0.960/3.648/1.551 ms
mininet>
```

4. H3 -> H1 Ping Failed

```
mininet> h3 ping -c 4 h1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
From 10.0.0.3 icmp_seq=1 Destination Host Unreachable
From 10.0.0.3 icmp_seq=2 Destination Host Unreachable
From 10.0.0.3 icmp_seq=3 Destination Host Unreachable
From 10.0.0.3 icmp_seq=4 Destination Host Unreachable

--- 10.0.0.1 ping statistics ---
4 packets transmitted, 0 received, +4 errors, 100% packet loss, time 3061ms
pipe 4
```

5. Flow Table Rules

```
mininet> sh ovs-ofctl dump-flows s1
 cookie=0x0, duration=21.163s, table=0, n_packets=5, n_bytes=378, idle_timeout=30, priority=10,dl_src=00:00:00:00:00:01,
dl_dst=00:00:00:00:00:02 actions=output:"s1-eth2"
 cookie=0x0, duration=21.161s, table=0, n_packets=5, n_bytes=378, idle_timeout=30, priority=10,dl_src=00:00:00:00:00:02,
dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"
 cookie=0x0, duration=7.820s, table=0, n_packets=5, n_bytes=210, idle_timeout=30, priority=10,dl_src=00:00:00:00:00:01,d
l_dst=00:00:00:00:00:03 actions=drop
 cookie=0x0, duration=576.632s, table=0, n_packets=65, n_bytes=4530, priority=0 actions=CONTROLLER:65535
mininet> sh ovs-ofctl dump-flows s1
 cookie=0x0, duration=32.036s, table=0, n_packets=5, n_bytes=378, idle_timeout=30, priority=10,dl_src=00:00:00:00:00:01,
dl_dst=00:00:00:00:00:02 actions=output:"s1-eth2"
 cookie=0x0, duration=32.034s, table=0, n_packets=5, n_bytes=378, idle_timeout=30, priority=10,dl_src=00:00:00:00:00:02,
dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"
 cookie=0x0, duration=18.693s, table=0, n_packets=5, n_bytes=210, idle_timeout=30, priority=10,dl_src=00:00:00:00:00:01,
dl_dst=00:00:00:00:00:03 actions=drop
 cookie=0x0, duration=587.505s, table=0, n_packets=66, n_bytes=4600, priority=0 actions=CONTROLLER:65535
mininet>
```

6. ‘iperf’ Test

```
mininet> h2 iperf -s &
------------------------------------------------------------
Server listening on TCP port 5001
TCP window size: 85.3 KByte (default)
------------------------------------------------------------
mininet> h1 iperf -c h2
------------------------------------------------------------
Client connecting to 10.0.0.2, TCP port 5001
TCP window size: 85.3 KByte (default)
------------------------------------------------------------
[  1] local 10.0.0.1 port 40070 connected with 10.0.0.2 port 5001 (icwnd/mss/irtt=14/1448/30000)
[ ID] Interval       Transfer     Bandwidth
[  1] 0.0000-10.0158 sec  31.1 GBytes  26.7 Gbits/sec
mininet>
```

GitHub Repository Link:

https://github.com/jashruth-k-a/SDN-Based-Access-Control-System

Conclusion: This project demonstrates the use of SDN for implementing dynamic and centralized access control. By using Ryu and Mininet, a whitelist-based system was successfully developed to allow communication between authorized hosts while blocking unauthorized ones.

The approach ensures efficient policy enforcement at the data plane level and highlights the flexibility and security advantages of SDN.